CAC Hub

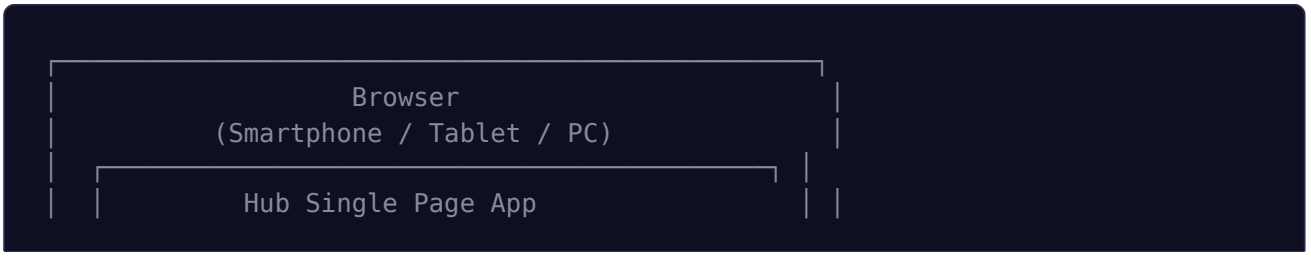
Technical Documentation — Central Dashboard for Pioneer CAC CD Autochanger Nodes

Contents

Project Description

The CAC Hub is a central dashboard for managing multiple Pioneer CAC CD autochanger units across the network. It connects via REST API and WebSocket to any number of CAC Controller nodes, providing a unified interface for monitoring and controlling all changers from a single location.

System Architecture



---

1. Concept and Design Principles

1.1 Decentralized Architecture

Each CAC Controller node runs as a standalone application on a Raspberry Pi, controlling exactly one Pioneer CD autochanger. The Hub is — every node works fully without it.

1.2 Hub as Aggregator

The Hub collects state information from all nodes and forwards it to Hub UI clients. It does not store CD data or libraries — those remain exclusively on the nodes.

1.3 Full Remote Access

Via the Proxy API, the Hub has to every function of each node: player control, library CRUD, playlists, scanner, MusicBrainz search, favorites, ratings, history, settings — everything.

1.4 API Key Authentication

All communication between Hub and nodes is authenticated via API key. The Hub sends the `X-API-Key` header with every proxy request and connects via `ws://node:port/?hub=1&apikey=<key>` for WebSocket.

---

2. Software Architecture

2.1 Technology Stack

Component	Technology	Version
Runtime	Node.js	≥ 18.x
Web Framework	Express	5.x
WebSocket	ws	8.x
Database	better-sqlite3	11.x
Frontend	Vanilla JS (SPA)	—
i18n	Custom solution	DE/EN

2.2 Project Structure

```
cac-controller-hub/
├── server.js                # Entry point (Port 4000)
├── package.json
├── cac-hub.service          # Systemd unit
├── src/
│   ├── database.js          # SQLite (Nodes, Settings)
│   ├── node-manager.js      # WebSocket connections to nodes
│   ├── routes.js            # Hub API + proxy routes
│   └── websocket.js         # WebSocket for Hub UI
├── public/
```

```
│   ├── index.html            # Hub SPA
│   ├── css/app.css           # Dark theme
│   └── js/
│       ├── app.js            # Frontend logic
│       └── i18n.js           # Translations (DE/EN)
└── data/
    └── cac-hub.db            # SQLite database
```

2.3 Module Overview

server.js — Entry Point

Initializes database, NodeManager, Express server, and WebSocket. Connects to all enabled nodes on startup. Graceful shutdown via SIGINT/SIGTERM.

database.js — SQLite Database

Stores node configuration and Hub settings:

```
nodes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT,                    -- Display name
  url TEXT NOT NULL,            -- e.g. http://192.168.1.100:3000
  api_key TEXT,                 -- API key for authentication
  room TEXT,                    -- Room/location
  model TEXT,                   -- Pioneer model (automatic)
  enabled INTEGER DEFAULT 1,   -- Enabled/disabled
  sort_order INTEGER DEFAULT 0,
  created_at TEXT,
  updated_at TEXT
)

settings (
  key TEXT PRIMARY KEY,
  value TEXT
)
```

Default settings: `hub_port`, `hub_name`, `language`, `reconnect_interval`, `health_check_interval`.

node-manager.js — Node Connection Manager

Core module for communication with all nodes:

- Connects via `ws://node:port/?hub=1&apikey=<key>` to each enabled node

- Receives `hubInit` message on connection (name, room, model, player state)
- Receives all broadcast events (playerState, scanProgress, playModeChange, etc.)
- Aggregates the state of all nodes in a central Map

- On connection loss, reconnection is attempted every 10 seconds (configurable)
- Node data is refreshed from the DB before reconnect (in case URL/key changed)

- Every 30 seconds (configurable), each node's status is queried via REST API
- Serves as backup in case WebSocket events are lost

- `proxyRequest(nodeId, method, path, body)` — forwards any API call to the node
- `proxyBinary(nodeId, path)` — forwards binary content (cover images)
- Automatically adds the `X-API-Key` header

- `nodeOnline` — Node connected
- `nodeOffline` — Connection lost
- `nodeState` — Full state received (hubInit)
- `nodeEvent` — Individual event from a node

routes.js — Hub API and Proxy

- `GET /api/hub/status` — Hub overview with all nodes
- CRUD for nodes: `GET/POST/PUT/DELETE /api/nodes`
- `POST /api/nodes/test` — Connection test to a node
- `GET/PUT /api/settings` — Hub settings

- `ALL /api/nodes/:id/proxy/{*path}` — Forwards any API call to the node
- `GET /api/nodes/:id/cover/{*path}` — Forwards cover images

The proxy is transparent: the Hub client calls e.g. `/api/nodes/1/proxy/library`, the Hub forwards this as `GET http://node:3000/api/library` with the API key.

websocket.js — Hub WebSocket

Manages WebSocket connections from Hub UI clients:

- On connection: sends `init` message with the current state of all nodes
- Forwards all events from NodeManager to all Hub clients
- Each message includes the `nodeId` so the client knows which node is affected

---

3. Frontend

3.1 Dashboard

The dashboard displays all configured nodes as cards in a responsive grid:

- : Green dot = online, red dot = offline
- : For each node, both players show mode, disc, and track
- : Play, Pause, Stop, and Next directly on the dashboard card
- : Opens the detail view for that node

3.2 Node Detail View

Full control of a selected node with four sub-tabs:

- Both players side by side (responsive: stacked on mobile devices)
- Cover image, disc title, artist, slot number
- Track number and track title
- Time display (real-time via WebSocket)
- Transport controls: Previous, Stop, Play, Pause, Next
- Volume slider
- Track list for the current CD (clickable to play)

- Play modes: Continuous, Gapless, Shuffle

- Searchable list of all CDs on the node
- Cover images (loaded via proxy)
- Detail modal with track list
- Load buttons for Player 1 and Player 2

- List of all playlists on the node
- Play button to start

- Enter start/end slot
- Start/abort scan
- Live progress bar via WebSocket

3.3 Settings

- : List of all nodes with status, delete button
- : Enter URL, API key, name, room; connection test; auto-fill of name/room/model
- : Name, port, language

3.4 Internationalization

Same as the node: `data-i18n` attributes, `t('key')` function, language toggle button. Approx. 80 translation keys for German and English.

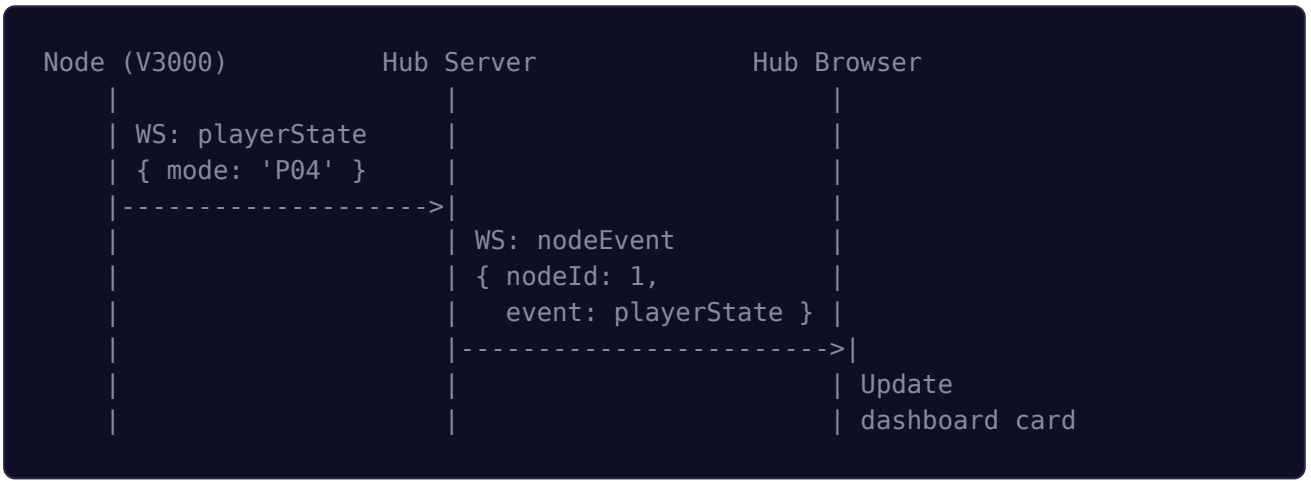
---

4. Data Flow

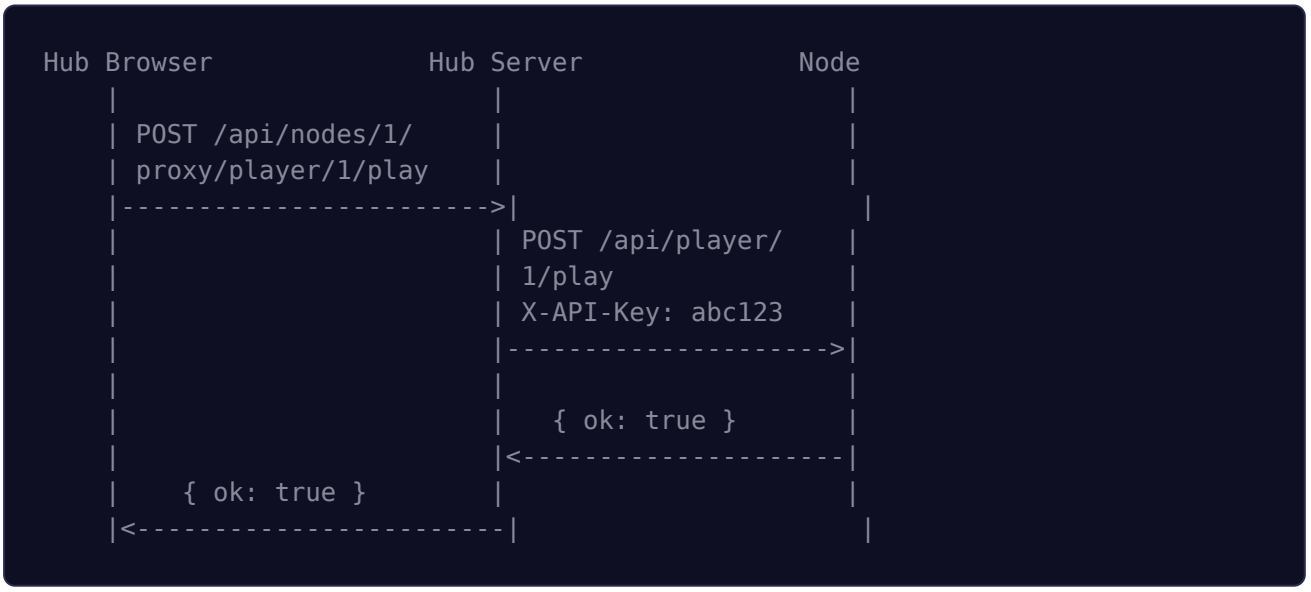
4.1 Hub Startup

```
1. Initialize database
2. Create NodeManager
3. Start Express server (Port 4000)
4. Start Hub WebSocket
5. For each enabled node:
   a. Open WebSocket connection
   b. Receive hubInit -> store state
   c. Start health check timer
```

4.2 Dashboard Update on Player Change



4.3 Proxy Command (e.g. pressing Play)



4.4 Loading Library

---

5. Database Details

5.1 Schema

The Hub database is minimalistic — it only stores node configurations and Hub settings. All CD data, playlists, favorites, etc. remain on the nodes.

5.2 WAL Mode

Write-Ahead Logging for better performance with concurrent reads and writes.

5.3 Default Settings

Key	Default	Description
`hub_port`	`4000`	HTTP port
`hub_name`	`CAC Hub`	Display name
`language`	`auto`	Language (auto/de/en)
`reconnect_interval`	`10000`	Reconnect interval (ms)
`health_check_interval`	`30000`	Health check interval (ms)

---

6. REST API Reference

6.1 Hub Endpoints

Method	Endpoint	Description
`GET`	`/api/hub/status`	Hub overview with all nodes and status
`GET`	`/api/nodes`	All nodes with connection status
`POST`	`/api/nodes`	Add node `{ url, api_key, name, room }`
`GET`	`/api/nodes/:id`	Get single node with status
`PUT`	`/api/nodes/:id`	Update node
`DELETE`	`/api/nodes/:id`	Remove node
`POST`	`/api/nodes/test`	Connection test `{ url, api_key }`
`GET`	`/api/settings`	Get Hub settings
`PUT`	`/api/settings`	Update Hub settings

6.2 Proxy Endpoints

Method	Endpoint	Description
`ALL`	`/api/nodes/:id/proxy/*`	Forward any API call to node
`GET`	`/api/nodes/:id/cover/*`	Load cover image from node

Examples:

- `GET /api/nodes/1/proxy/library` → `GET http://node:3000/api/library`
- `POST /api/nodes/1/proxy/player/1/play` → `POST http://node:3000/api/player/1/play`
- `PUT /api/nodes/1/proxy/library/42` → `PUT http://node:3000/api/library/42`
- `GET /api/nodes/1/cover/cover_42.jpg` → `GET http://node:3000/covers/cover_42.jpg`

6.3 WebSocket Messages

Type	Direction	Description

`init`	Hub → Client	Initial state of all nodes
`nodeOnline`	Hub → Client	Node connected
`nodeOffline`	Hub → Client	Node disconnected
`nodeState`	Hub → Client	Full state of a node
`nodeEvent`	Hub → Client	Forwarded event (playerState, scanProgress, etc.)

---

7. Deployment

7.1 Systemd Service

```
[Unit]
Description=CAC Hub - Central Dashboard for Pioneer CD Autochanger Nodes
After=network.target

[Service]
Type=simple
User=pi
WorkingDirectory=/opt/cac-controller-hub
ExecStart=/usr/bin/node server.js
Restart=always
RestartSec=5
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

7.2 Hub and Node on the Same Pi

Both applications can run simultaneously on the same Raspberry Pi:

- Node on port 3000 (default)
- Hub on port 4000 (default)
- Add node to Hub with URL `http://localhost:3000`

7.3 Network Requirements

- All nodes must be reachable via HTTP from the Hub Pi

- Default ports: Node 3000, Hub 4000
- Recommended: Static IP addresses or DNS for all Pis

---

8. Security

8.1 API Key Authentication

- Each node generates a 32-character alphanumeric API key
- The Hub stores the key encrypted in its SQLite database
- All proxy requests include the `X-API-Key` header
- WebSocket connections require the key as a query parameter

8.2 Recommendations

- Operate only on the local network
- No port forwarding to the internet
- Use strong, random API keys (the node UI automatically generates secure keys)
- If needed: Nginx as reverse proxy with TLS

---

9. Known Limitations

1. : The Hub does not store CD data. If the connection to a node is lost, its data is no longer visible in the Hub.
2. : The Hub cannot play audio from one node on another device. Each node has its own analog audio output.
3. : Playlists can only contain CDs from a single node. A playlist spanning multiple changers is not supported.
4. : Communication between Hub and nodes is unencrypted (intended for LAN use only).

10. License

MIT License — Copyright © 2025 Dirk Jensen — see LICENSE.